

Gestion_DPI_backend Documentation

version

**2025, Hammou Manel, Bouameur Besmala Aicha, Taibi Meriem, Aouissi Bouchra Souad,
Messadia Mohamed Ishak, Malek Yasmine**

janvier 05, 2025

Sommaire

Gestion_DPI_backend documentation	1
Index	29
Index des modules Python	35

Gestion_DPI_backend documentation

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

Tests unitaires pour les vues de l'application backendapp.

Ce fichier contient les tests associés aux vues du backendapp. Les tests visent à vérifier le bon fonctionnement des API créées pour interagir avec les modèles et les données de l'application.

Ces tests sont utilisés pour s'assurer que les vues du backend fonctionnent comme prévu et renvoient les bonnes informations ou des erreurs appropriées.

```
class backendapp.tests.OrdonnanceModelTestCase (methodName='runTest')
```

Bases : **TestCase**

Test des fonctionnalités liées au modèle Ordonnance.

Cette classe regroupe les tests pour vérifier : - La création d'une ordonnance valide. - La bonne association d'un dossier patient avec un patient et une consultation.

setUp ()

Préparation des données de test avant chaque test.

Cette méthode crée un médecin, un patient, un dossier patient et une consultation nécessaires pour les tests de l'ordonnance.

test_dossier_patient_association ()

Vérifier que le dossier patient est bien associé à son patient.

Ce test assure que la relation entre le patient et son dossier patient est correctement établie.

test_ordonnance_creation ()

Tester la création d'une ordonnance valide.

Ce test vérifie que l'ordonnance est correctement créée et que la relation entre l'ordonnance, le dossier patient et la consultation est correcte.

```
class backendapp.serializers.AdministrateurSerializer (*args, **kwargs)
```

Bases : **ModelSerializer**

Sérialiseur pour le rôle d'Administrateur. Hérite des champs du modèle User.

```
class Meta
```

Bases : **object**

fields = '__all__'

model

alias de **Administrateur**

```
class backendapp.serializers.BilanBiologiqueSerializer (*args, **kwargs)
```

Bases : **ModelSerializer**

Sérialiseur pour le bilan biologique. Gère les informations relatives aux bilans biologiques effectués sur le patient.

```
class Meta
```

Bases : **object**

fields = '__all__'

model

alias de **BilanBiologique**

```
class backendapp.serializers.BilanRadiologiqueSerializer (*args, **kwargs)
```

Bases : **ModelSerializer**

Sérialiseur pour le bilan radiologique. Gère les informations relatives aux bilans radiologiques effectués sur le patient.

```
class Meta
```

Bases : **object**

fields = ' __all__ '

model

alias de **BilanRadiologique**

class backendapp.serializers.**ConsultationSerializer** (*args, **kwargs)

Bases : **ModelSerializer**

Sérialiseur pour la consultation d'un patient. Gère les informations de la consultation ainsi que le médecin consultant.

class **Meta**

Bases : **object**

fields = ' __all__ '

model

alias de **Consultation**

class backendapp.serializers.**DossierPatientSerializer** (*args, **kwargs)

Bases : **ModelSerializer**

Sérialiseur pour le modèle DossierPatient, liant un patient spécifique au dossier.

class **Meta**

Bases : **object**

fields = ' __all__ '

model

alias de **DossierPatient**

class backendapp.serializers.**InfirmierSerializer** (*args, **kwargs)

Bases : **ModelSerializer**

Sérialiseur pour le rôle d'Infirmier. Gère les informations spécifiques à l'infirmier.

class **Meta**

Bases : **object**

fields = ' __all__ '

model

alias de **Infirmier**

class backendapp.serializers.**LaborantinSerializer** (*args, **kwargs)

Bases : **ModelSerializer**

Sérialiseur pour le rôle de Laborantin. Gère les informations spécifiques au laborantin.

class **Meta**

Bases : **object**

fields = ' __all__ '

model

alias de **Laborantin**

class backendapp.serializers.**MedecinSerializer** (*args, **kwargs)

Bases : **ModelSerializer**

Sérialiseur pour le rôle de Médecin. Gère les informations spécifiques au médecin.

class **Meta**

Bases : **object**

fields = ' __all__ '

model

alias de **Medecin**

`class backendapp.serializers.MedicamentsSerializer(*args, **kwargs)`

Bases : **ModelSerializer**

Sérialiseur pour les médicaments prescrits. Gère les informations des médicaments associés à un soin ou une ordonnance.

`class Meta`

Bases : **object**

fields = ' __all__ '

model

alias de **Medicament**

`class backendapp.serializers.OrdonnaceSerializer(*args, **kwargs)`

Bases : **ModelSerializer**

Sérialiseur pour les ordonnances prescrites. Gère les informations des ordonnances.

`class Meta`

Bases : **object**

fields = ' __all__ '

model

alias de **Ordonnance**

`class backendapp.serializers.PatientSerializer(*args, **kwargs)`

Bases : **ModelSerializer**

Sérialiseur pour le rôle de Patient. Gère la relation avec le médecin traitant et le dossier du patient. Les informations du médecin traitant et du dossier sont sérialisées de manière imbriquée.

`class Meta`

Bases : **object**

fields = ' __all__ '

model

alias de **Patient**

`class backendapp.serializers.RadiologueSerializer(*args, **kwargs)`

Bases : **ModelSerializer**

Sérialiseur pour le rôle de Radiologue. Gère les informations spécifiques au radiologue.

`class Meta`

Bases : **object**

fields = ' __all__ '

model

alias de **Radiologue**

`class backendapp.serializers.SoinsSerializer(*args, **kwargs)`

Bases : **ModelSerializer**

Sérialiseur pour les soins administrés au patient. Gère toutes les informations liées aux soins médicaux.

```

class Meta
    Bases : object

    fields = '__all__'

    model
        alias de Soins

class backendapp.serializers.UserSerializer (*args, **kwargs)
    Bases : ModelSerializer
    S rialiseur pour le mod le utilisateur de base. G re les champs communs   tous les r les d'utilisateur. Le champ
    mot de passe est  crit uniquement, il ne sera pas retourn  lors de la s rialisation.

class Meta
    Bases : object

    fields = ['id', 'username', 'first_name', 'last_name', 'email', 'role', 'date_naissance', 'adresse',
    'numero_telephone']

    model
        alias de User

class backendapp.apps.BackendappConfig (app_name, app_module)
    Bases : AppConfig
    Configuration de l'application "backendapp" dans le projet Django.
    Cette classe permet de d finir les param tres de base de l'application, tels que : - default_auto_field: sp cifie le
    type de champ   utiliser pour les cl s primaires automatiques. - name: d finit le nom de l'application.

    default_auto_field = 'django.db.models.BigAutoField'

    name = 'backendapp'

class backendapp.admin.AdministrateurAdmin (model, admin_site)
    Bases : UserAdmin
    Configuration de l'interface d'administration pour le mod le Administrateur. G re les utilisateurs ayant le r le
    d'administrateur.

    add_fieldsets = ((None, {'classes': ('wide',), 'fields': ('username', 'usable_password', 'password1',
    'password2')}), (None, {'fields': ('email', 'first_name', 'last_name', 'date_naissance', 'adresse',
    'numero_telephone')}))

    fieldsets = ((None, {'fields': ('username', 'password')}), ('Personal info', {'fields': ('first_name', 'last_name',
    'email')}), ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions')}), ('Important
    dates', {'fields': ('last_login', 'date_joined')}), (None, {'fields': ('date_naissance', 'adresse', 'numero_telephone')}))

    list_display = ('username', 'role', 'email', 'first_name', 'last_name', 'date_naissance', 'numero_telephone')
    property media

    model
        alias de Administrateur

    ordering = ('username',)

    save_model (request, obj, form, change)
        D finit le r le de l'utilisateur comme "Administrateur" lors de la sauvegarde.

    search_fields = ('username', 'email')

class backendapp.admin.InfirmierAdmin (model, admin_site)
    Bases : UserAdmin

```


Configuration de l'interface d'administration pour le modèle Infirmier. Gère les utilisateurs ayant le rôle d'infirmier.

```
add_fieldsets = ((None, {'classes': ('wide',), 'fields': ('username', 'usable_password', 'password1', 'password2')}), (None, {'fields': ('email', 'first_name', 'last_name', 'date_naissance', 'adresse', 'numero_telephone')}))
```

```
fieldsets = ((None, {'fields': ('username', 'password')}), ('Personal info', {'fields': ('first_name', 'last_name', 'email')}), ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions')}), ('Important dates', {'fields': ('last_login', 'date_joined')}), (None, {'fields': ('date_naissance', 'adresse', 'numero_telephone')}))
```

```
list_display = ('username', 'role', 'email', 'first_name', 'last_name', 'date_naissance', 'numero_telephone')
```

```
property media
```

```
model
```

```
alias de Infirmier
```

```
ordering = ('username',)
```

```
save_model(request, obj, form, change)
```

```
Définit le rôle de l'utilisateur comme "Infirmier" lors de la sauvegarde.
```

```
search_fields = ('username', 'email')
```

```
class backendapp.admin.LaborantinAdmin(model, admin_site)
```

```
Bases : UserAdmin
```

Configuration de l'interface d'administration pour le modèle Laborantin. Gère les utilisateurs ayant le rôle de laborantin.

```
add_fieldsets = ((None, {'classes': ('wide',), 'fields': ('username', 'usable_password', 'password1', 'password2')}), (None, {'fields': ('email', 'first_name', 'last_name', 'date_naissance', 'adresse', 'numero_telephone')}))
```

```
fieldsets = ((None, {'fields': ('username', 'password')}), ('Personal info', {'fields': ('first_name', 'last_name', 'email')}), ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions')}), ('Important dates', {'fields': ('last_login', 'date_joined')}), (None, {'fields': ('date_naissance', 'adresse', 'numero_telephone')}))
```

```
list_display = ('username', 'role', 'email', 'first_name', 'last_name', 'date_naissance', 'numero_telephone')
```

```
property media
```

```
model
```

```
alias de Laborantin
```

```
ordering = ('username',)
```

```
save_model(request, obj, form, change)
```

```
Définit le rôle de l'utilisateur comme "Laborantin" lors de la sauvegarde.
```

```
search_fields = ('username', 'email')
```

```
class backendapp.admin.MedecinAdmin(model, admin_site)
```

```
Bases : UserAdmin
```

Configuration de l'interface d'administration pour le modèle Medecin. Gère les utilisateurs ayant le rôle de médecin.

```
add_fieldsets = ((None, {'classes': ('wide',), 'fields': ('username', 'usable_password', 'password1', 'password2')}), (None, {'fields': ('email', 'first_name', 'last_name', 'date_naissance', 'adresse', 'numero_telephone')}))
```

```
fieldsets = ((None, {'fields': ('username', 'password')}), ('Personal info', {'fields': ('first_name', 'last_name', 'email')}), ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions')}), ('Important dates', {'fields': ('last_login', 'date_joined')}), (None, {'fields': ('date_naissance', 'adresse', 'numero_telephone')}))
```

```
list_display = ('username', 'role', 'email', 'first_name', 'last_name', 'date_naissance', 'numero_telephone')
```

property **media**

model

alias de **Medecin**

```
ordering = ('username',)
```

```
save_model(request, obj, form, change)
```

Définit le rôle de l'utilisateur comme "Medecin" lors de la sauvegarde.

```
search_fields = ('username', 'email')
```

```
class backendapp.admin.PatientAdmin(model, admin_site)
```

Bases : **UserAdmin**

Configuration de l'interface d'administration pour le modèle Patient. Personnalise l'affichage, les champs disponibles et les actions possibles.

```
add_fieldsets = ((None, {'classes': ('wide',), 'fields': ('username', 'usable_password', 'password1', 'password2')}), (None, {'fields': ('email', 'first_name', 'last_name', 'date_naissance', 'adresse', 'numero_telephone', 'nss', 'mutuelle', 'medecin_traitant')}))
```

```
fieldsets = ((None, {'fields': ('username', 'password')}), ('Personal info', {'fields': ('first_name', 'last_name', 'email')}), ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions')}), ('Important dates', {'fields': ('last_login', 'date_joined')}), (None, {'fields': ('date_naissance', 'adresse', 'numero_telephone', 'nss', 'mutuelle', 'medecin_traitant')}))
```

form

alias de **PatientAdminForm**

```
list_display = ('username', 'role', 'email', 'first_name', 'last_name', 'date_naissance', 'numero_telephone', 'nss', 'mutuelle', 'medecin_traitant')
```

property **media**

model

alias de **Patient**

```
ordering = ('nss',)
```

```
save_model(request, obj, form, change)
```

Méthode appelée lors de la sauvegarde d'un objet Patient dans l'interface d'administration. Personnalise la sauvegarde en définissant le username comme le NSS, en créant un DossierPatient par défaut, et en assignant les rôles et champs spécifiques.

```
search_fields = ('nss',)
```

```
class backendapp.admin.PatientAdminForm (data=None, files=None, auto_id='id_%s', prefix=None, initial=None, error_class=<class 'django.forms.utils.ErrorList'>, label_suffix=None, empty_permitted=False, instance=None, use_required_attribute=None, renderer=None)
```

Bases : **ModelForm**

Formulaire personnalisé pour le modèle Patient dans l'interface d'administration. Permet d'ajouter des champs spécifiques tels que le NSS, la mutuelle et le médecin traitant.

```
class Meta
```

Bases : **object**

Méta-données associées au formulaire PatientAdminForm. Indique que ce formulaire est lié au modèle Patient.

fields = `'__all__'`

model

alias de **Patient**

```
base_fields = {'adresse': <django.forms.fields.CharField object>, 'date_joined':
<django.forms.fields.DateTimeField object>, 'date_naissance': <django.forms.fields.DateField object>,
'dossier_patient': <django.forms.models.ModelChoiceField object>, 'email': <django.forms.fields.EmailField
object>, 'first_name': <django.forms.fields.CharField object>, 'groups':
<django.forms.models.ModelMultipleChoiceField object>, 'is_active': <django.forms.fields.BooleanField object>,
'is_staff': <django.forms.fields.BooleanField object>, 'is_superuser': <django.forms.fields.BooleanField object>,
'last_login': <django.forms.fields.DateTimeField object>, 'last_name': <django.forms.fields.CharField object>,
'medecin_traitant': <django.forms.models.ModelChoiceField object>, 'mutuelle': <django.forms.fields.CharField
object>, 'nss': <django.forms.fields.CharField object>, 'numero_telephone': <django.forms.fields.CharField
object>, 'password': <django.forms.fields.CharField object>, 'role': <django.forms.fields.TypedChoiceField object>,
'telephone_urgence': <django.forms.fields.CharField object>, 'user_permissions':
<django.forms.models.ModelMultipleChoiceField object>, 'username': <django.forms.fields.CharField object>}
```

```
declared_fields = {'medecin_traitant': <django.forms.models.ModelChoiceField object>, 'mutuelle':
<django.forms.fields.CharField object>, 'nss': <django.forms.fields.CharField object>}
```

property media

Return all media required to render the widgets on this form.

```
class backendapp.admin.RadiologueAdmin (model, admin_site)
```

Bases : **UserAdmin**

Configuration de l'interface d'administration pour le modèle Radiologue. Gère les utilisateurs ayant le rôle de radiologue.

```
add_fieldsets = ((None, {'classes': ('wide',), 'fields': ('username', 'usable_password', 'password1',
'password2')}), (None, {'fields': ('email', 'first_name', 'last_name', 'date_naissance', 'adresse',
'numero_telephone')}))
```

```
fieldsets = ((None, {'fields': ('username', 'password')}), ('Personal info', {'fields': ('first_name', 'last_name',
'email')}), ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions')}), ('Important
dates', {'fields': ('last_login', 'date_joined')}), (None, {'fields': ('date_naissance', 'adresse', 'numero_telephone')}))
```

```
list_display = ('username', 'role', 'email', 'first_name', 'last_name', 'date_naissance', 'numero_telephone')
```

property media

model

alias de **Radiologue**

```
ordering = ('username',)
```

```
save_model (request, obj, form, change)
```

Définit le rôle de l'utilisateur comme "Radiologue" lors de la sauvegarde.

```
search_fields = ('username', 'email')
```

```
class backendapp.models.Administrateur (*args, **kwargs)
```

Bases : **User**

Modèle spécifique pour les administrateurs. Hérite de la classe User.

```
exception DoesNotExist
```

Bases : **DoesNotExist**

```
exception MultipleObjectsReturned
```

Bases : **MultipleObjectsReturned**

user_ptr

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Restaurant.place` is a `ForwardOneToOneDescriptor` instance.

user_ptr_id

```
class backendapp.models.BilanBiologique(*args, **kwargs)
```

Bases : **Model**

Modèle représentant un bilan biologique. : noindex: .. attribute:: dossier_patient

Référence au dossier du patient.

type: ForeignKey

laborantin

Référence au laborantin ayant effectué le bilan.

Type: ForeignKey

date_examen

Date de l'examen.

Type: datetime

graphe

Graphe associé au bilan (facultatif).

Type: ImageField

glycemie

Glycémie mesurée (facultatif).

Type: DecimalField

pression_arterielle

Pression artérielle mesurée (facultatif).

Type: str

cholesterol

Cholestérol mesuré (facultatif).

Type: DecimalField

numero_consultation

Numéro de consultation associé.

Type: int

exception **DoesNotExist**

Bases : **ObjectDoesNotExist**

exception **MultipleObjectsReturned**

Bases : **MultipleObjectsReturned**

cholesterol

noindex:

date_examen

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

dossier_patient

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

dossier_patient_id

get_next_by_date_examen (*, field=<django.db.models.fields.DateTimeField: date_examen>, is_next=True, **kwargs)

get_previous_by_date_examen (*, field=<django.db.models.fields.DateTimeField: date_examen>, is_next=False, **kwargs)

glycemie

noindex:

graphe

Just like the FileDescriptor, but for ImageFields. The only difference is assigning the width/height to the width_field/height_field, if appropriate.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

laborantin

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

laborantin_id

numero_consultation

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

pression_arterielle

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

```
class backendapp.models.BilanRadiologique(*args, **kwargs)
```

Bases : **Model**

Modèle représentant un bilan radiologique. : noindex: .. attribute:: dossier_patient

Référence au dossier du patient.

type: ForeignKey

radiologue

Référence au radiologue ayant effectué le bilan.

Type: ForeignKey

compte_rendu

Compte rendu de l'examen.

Type: TextField

images

Images associées au bilan (facultatif).

Type: ImageField

date_examen

Date de l'examen.

Type: datetime

numero_consultation

Numéro de consultation associé.

Type: int

exception **DoesNotExist**

Bases : **ObjectDoesNotExist**

exception **MultipleObjectsReturned**

Bases : **MultipleObjectsReturned**

compte_rendu

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

date_examen

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

dossier_patient

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

dossier_patient_id

get_next_by_date_examen (*, field=<django.db.models.fields.DateTimeField: date_examen>, is_next=True, **kwargs)

get_previous_by_date_examen (*, field=<django.db.models.fields.DateTimeField: date_examen>, is_next=False, **kwargs)

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

images

Just like the FileDescriptor, but for ImageFields. The only difference is assigning the width/height to the width_field/height_field, if appropriate.

numero_consultation

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

radiologue

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

radiologue_id

```
class backendapp.models.Consultation(*args, **kwargs)
```

Bases : **Model**

Modèle représentant une consultation médicale. : noindex: .. attribute:: dossier_patient

Référence au dossier du patient.

type: ForeignKey

numero_consultation

Numéro unique de la consultation.

Type: int

date_consultation

Date de la consultation.

Type: datetime

bilan_prescrit

Bilan prescrit lors de la consultation.

Type: MultiSelectField

resume

Résumé de la consultation.

Type: TextField

medecinConsultant

Médecin ayant réalisé la consultation.

Type: ForeignKey

BILAN_CHOICES = [('bilan biologique', 'Bilan Biologique'), ('bilan radiologique', 'Bilan Radiologique'), ('Aucun bilan', 'Aucun Bilan')]

exception DoesNotExist

Bases : **ObjectDoesNotExist**

exception MultipleObjectsReturned

Bases : **MultipleObjectsReturned**

bilan_prescrit

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

date_consultation

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

dossier_patient

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

dossier_patient_id**get_bilan_prescrit_display()****get_bilan_prescrit_list()**

get_next_by_date_consultation (*, field=<django.db.models.fields.DateTimeField: date_consultation>, is_next=True, **kwargs)

get_previous_by_date_consultation (*, field=<django.db.models.fields.DateTimeField: date_consultation>, is_next=False, **kwargs)

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

medecinConsultant

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

medecinConsultant_id**numero_consultation**

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

ordonnance_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

resume

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.


```
class backendapp.models.DossierPatient(*args, **kwargs)
    Bases : Model
    Modèle représentant le dossier médical d'un patient. : noindex: .. attribute:: etat
        État général du dossier (facultatif).
            type: str
```

antécédents
Antécédents médicaux du patient (facultatif).
Type: TextField

```
exception DoesNotExist
    Bases : ObjectDoesNotExist
```

```
exception MultipleObjectsReturned
    Bases : MultipleObjectsReturned
```

antécédents
A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

bilanbiologique_set
Accessor to the related objects manager on the reverse side of a many-to-one relation.
In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.
Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

bilanradiologique_set
Accessor to the related objects manager on the reverse side of a many-to-one relation.
In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.
Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

consultation_set
Accessor to the related objects manager on the reverse side of a many-to-one relation.
In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.
Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

etat
A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

id
A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

ordonnance_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

patient

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Place.restaurant is a ReverseOneToOneDescriptor instance.

soins_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

```
class backendapp.models.Infirmier(*args, **kwargs)
```

Bases : **User**

Modèle spécifique pour les infirmiers. Hérite de la classe User.

exception **DoesNotExist**

Bases : **DoesNotExist**

exception **MultipleObjectsReturned**

Bases : **MultipleObjectsReturned**

patients

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

user_ptr

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Restaurant.place is a ForwardOneToOneDescriptor instance.

user_ptr_id

```
class backendapp.models.Laborantin(*args, **kwargs)
    Bases : User
    Modèle spécifique pour les laborantins. Hérite de la classe User.
```

```
exception DoesNotExist
    Bases : DoesNotExist
```

```
exception MultipleObjectsReturned
    Bases : MultipleObjectsReturned
```

patients

Accessor to the related objects manager on the reverse side of a many-to-one relation.
In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.
Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

user_ptr

Accessor to the related object on the forward side of a one-to-one relation.
In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Restaurant.place is a ForwardOneToOneDescriptor instance.

user_ptr_id

```
class backendapp.models.Medecin(*args, **kwargs)
    Bases : User
    Modèle spécifique pour les médecins. Hérite de la classe User.
```

```
exception DoesNotExist
    Bases : DoesNotExist
```

```
exception MultipleObjectsReturned
    Bases : MultipleObjectsReturned
```

consultation_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.
In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.
Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

patients

Accessor to the related objects manager on the reverse side of a many-to-one relation.
In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.
Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

user_ptr

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Restaurant.place` is a `ForwardOneToOneDescriptor` instance.

user_ptr_id

```
class backendapp.models.Medicament(*args, **kwargs)
```

Bases : **Model**

Modèle représentant un médicament prescrit. : noindex: .. attribute:: ordonnance

Référence à l'ordonnance associée.

type: `ForeignKey`

nom

Nom du médicament.

Type: `str`

dose

Dosage du médicament.

Type: `str`

duree

Durée pendant laquelle le médicament doit être pris.

Type: `str`

exception **DoesNotExist**

Bases : **ObjectDoesNotExist**

exception **MultipleObjectsReturned**

Bases : **MultipleObjectsReturned**

dose

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

duree

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

nom

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = *<django.db.models.manager.Manager object>*

ordonnance

Accessor to the related object on the forward side of a many-to-one or one-to-one (via `ForwardOneToOneDescriptor` subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

ordonnance_id

```
class backendapp.models.Ordonnance (*args, **kwargs)
```

Bases : **Model**

Modèle représentant une ordonnance médicale. : noindex: .. attribute:: dossier_patient

Référence au dossier du patient.

type: ForeignKey

consultation

Référence à la consultation associée.

Type: ForeignKey

exception **DoesNotExist**

Bases : **ObjectDoesNotExist**

exception **MultipleObjectsReturned**

Bases : **MultipleObjectsReturned**

consultation

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

consultation_id

dossier_patient

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

dossier_patient_id

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

medicaments

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

objects = <django.db.models.manager.Manager object>

```
class backendapp.models.Patient(*args, **kwargs)
```

Bases : **User**

Modèle spécifique pour les patients. : noindex: .. attribute:: nss

Numéro de sécurité sociale unique.

type: str

medecin_traitant

Médecin traitant associé au patient.

Type: ForeignKey

mutuelle

Nom de la mutuelle du patient (facultatif).

Type: str

dossier_patient

Référence au dossier du patient.

Type: OneToOneField

telephone_urgence

Numéro de téléphone d'urgence (facultatif).

Type: str

exception **DoesNotExist**

Bases : **DoesNotExist**

exception **MultipleObjectsReturned**

Bases : **MultipleObjectsReturned**

dossier_patient

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Restaurant.place is a ForwardOneToOneDescriptor instance.

dossier_patient_id

medecin_traitant

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

medecin_traitant_id

mutuelle

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

nss

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

telephone_urgence

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

user_ptr

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Restaurant.place` is a `ForwardOneToOneDescriptor` instance.

user_ptr_id

```
class backendapp.models.Radiologue(*args, **kwargs)
```

Bases : **User**

Modèle spécifique pour les radiologues. Hérite de la classe User.

exception **DoesNotExist**

Bases : **DoesNotExist**

exception **MultipleObjectsReturned**

Bases : **MultipleObjectsReturned**

patients

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

user_ptr

Accessor to the related object on the forward side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Restaurant.place` is a `ForwardOneToOneDescriptor` instance.

user_ptr_id

```
class backendapp.models.Soins(*args, **kwargs)
```

Bases : **Model**

Modèle représentant les soins administrés à un patient. : noindex: .. attribute:: dossier_patient

Référence au dossier du patient.

type: ForeignKey

infirmier

Référence à l'infirmier ayant administré les soins.

Type: ForeignKey

observation_etat_patient

Observations sur l'état du patient.

Type: TextField

medicament_pris

Indique si un médicament a été pris.

Type: bool

description_soins

Description des soins administrés.

Type: TextField

date_soin

Date des soins.

Type: datetime

exception **DoesNotExist**

Bases : **ObjectDoesNotExist**

exception **MultipleObjectsReturned**

Bases : **MultipleObjectsReturned**

date_soin

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

description_soins

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

dossier_patient

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

dossier_patient_id

get_next_by_date_soin (*, field=<django.db.models.fields.DateTimeField: date_soin>, is_next=True, **kwargs)

get_previous_by_date_soin (*, field=<django.db.models.fields.DateTimeField: date_soin>, is_next=False, **kwargs)

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

infirmier

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```


`Child.parent` is a `ForwardManyToOneDescriptor` instance.

`infirmier_id`

`medicament_pris`

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

`objects` = *<django.db.models.manager.Manager object>*

`observation_etat_patient`

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

```
class backendapp.models.User(*args, **kwargs)
```

Bases : **`AbstractUser`**

Modèle personnalisé d'utilisateur. : noindex: .. attribute:: role

Le rôle de l'utilisateur parmi les choix disponibles (administrateur, patient, etc.).

`type:` str

`date_naissance`

Date de naissance de l'utilisateur (peut être vide pour les super-utilisateurs).

`Type:` date

`adresse`

Adresse de l'utilisateur.

`Type:` str

`numero_telephone`

Numéro de téléphone de l'utilisateur.

`Type:` str

exception **`DoesNotExist`**

Bases : **`ObjectDoesNotExist`**

exception **`MultipleObjectsReturned`**

Bases : **`MultipleObjectsReturned`**

`ROLE_CHOICES` = (('Administrateur', 'administrateur'), ('Patient', 'patient'), ('Medecin', 'medecin'), ('Infirmier', 'infirmier'), ('Laborantin', 'laborantin'), ('Radiologue', 'radiologue'))

`administrateur`

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Place.restaurant` is a `ReverseOneToOneDescriptor` instance.

`adresse`

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

`auth_token`

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Place.restaurant is a ReverseOneToOneDescriptor instance.

date_joined

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

date_naissance

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

email

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

first_name

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

```
get_next_by_date_joined(*, field=<django.db.models.fields.DateTimeField:
date_joined>, is_next=True, **kwargs)
```

```
get_previous_by_date_joined(*, field=<django.db.models.fields.DateTimeField:
date_joined>, is_next=False, **kwargs)
```

```
get_role_display(*, field=<django.db.models.fields.CharField: role>)
```

groups

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation. In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

Pizza.toppings and Topping.pizzas are ManyToManyDescriptor instances.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

infirmier

Accessor to the related object on the reverse side of a one-to-one relation. In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Place.restaurant is a ReverseOneToOneDescriptor instance.

is_active

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

is_staff

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

is_superuser

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

laborantin

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Place.restaurant is a ReverseOneToOneDescriptor instance.

last_login

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

last_name

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

logentry_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToManyDescriptor instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

medecin

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Place.restaurant is a ReverseOneToOneDescriptor instance.

numero_telephone

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

password

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

patient

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

Place.restaurant is a ReverseOneToOneDescriptor instance.

radiologue

Accessor to the related object on the reverse side of a one-to-one relation.

In the example:

```
class Restaurant(Model):
    place = OneToOneField(Place, related_name='restaurant')
```

`Place.restaurant` is a `ReverseOneToOneDescriptor` instance.

role

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

user_permissions

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation. In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

username

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

Vues de l'application backendapp.

Ce fichier contient les vues de l'application backendapp, qui sont responsables de la gestion des différentes requêtes HTTP pour récupérer les informations liées aux patients, consultations, bilans radiologiques et ordonnances.

Les vues incluent la récupération des bilans radiologiques, des ordonnances et des informations relatives à la consultation, ainsi que la validation des informations du patient.

Les réponses sont formatées en JSON, et les erreurs sont gérées de manière appropriée en renvoyant des codes d'état HTTP pertinents.

`backendapp.views.Faire_soin(request, *args, **kwargs)`

Crée un soin pour un patient donné.

Cette fonction permet de créer un soin pour un patient en utilisant son numéro de sécurité sociale (NSS), les observations sur l'état du patient, les médicaments administrés, la description du soin, ainsi que l'infirmier en charge (optionnel).

Paramètres:

`request (Request)`: La requête HTTP contenant les informations nécessaires à la création du soin.

Retour:

Response: La réponse HTTP contenant un message de succès ou un message d'erreur.

`backendapp.views.check_consultation_existence(request, *args, **kwargs)`

Vérifie l'existence d'une consultation pour un patient, et si un bilan radiologique a été prescrit.

Cette fonction prend le NSS et le numéro de consultation en entrée, et elle vérifie si : - Le patient existe dans la base de données. - Le dossier patient existe pour ce patient. - La consultation existe avec le numéro donné. - Le "bilan radiologique" est prescrit dans les bilans de la consultation.

Paramètres:

`request (Request)`: La requête HTTP contenant les informations de NSS et de numéro de consultation.

Retour:

Response: La réponse HTTP indiquant si la consultation existe, si un bilan radiologique est prescrit ou si des erreurs se sont produites.

`backendapp.views.check_consultation_existence_bilan_biologique(request, *args, **kwargs)`

Vérifie l'existence d'une consultation pour un patient, et si un bilan biologique a été prescrit.

Cette fonction prend le NSS et le numéro de consultation en entrée, et elle vérifie si : - Le patient existe dans la base de données. - Le dossier patient existe pour ce patient. - La consultation existe avec le numéro donné. - Le "bilan biologique" est prescrit dans les bilans de la consultation.

Paramètres:

`request (Request)`: La requête HTTP contenant les informations de NSS et de numéro de consultation.

Retour:

Response: La réponse HTTP indiquant si la consultation existe, si un bilan biologique est prescrit ou si des erreurs se sont produites.

`backendapp.views.creer_bilan_biologique` (request, *args, **kwargs)

Crée un bilan biologique pour un patient à partir des données fournies.

Cette fonction permet de créer un bilan biologique pour un patient donné. Elle attend des informations telles que le NSS du patient, l'ID du laborantin qui effectue l'examen, la date de l'examen, ainsi que des résultats de tests médicaux comme la glycémie, la pression artérielle, le cholestérol et un graphique (image). Elle vérifie également si un bilan biologique existe déjà pour ce patient et ce numéro de consultation.

Paramètres:

request (Request): La requête HTTP contenant les données nécessaires pour créer le bilan biologique.

Retour:

Response: La réponse HTTP avec un message de succès ou d'erreur.

`backendapp.views.creer_bilan_radiologique` (request, *args, **kwargs)

Crée un bilan radiologique pour un patient à partir des données fournies.

Cette fonction permet de créer un bilan radiologique pour un patient donné. Elle attend des informations telles que le NSS du patient, l'ID du radiologue qui a effectué l'examen, le compte-rendu de l'examen, la date de l'examen, ainsi qu'une image radiographique. Elle vérifie également si un bilan radiologique existe déjà pour ce patient et ce numéro de consultation.

Paramètres:

request (Request): La requête HTTP contenant les données nécessaires pour créer le bilan radiologique.

Retour:

Response: La réponse HTTP avec un message de succès ou d'erreur.

`backendapp.views.creer_consultation` (request, *args, **kwargs)

Crée une nouvelle consultation pour un patient donné.

Cette fonction permet de créer une consultation pour un patient à partir de son numéro de sécurité sociale (NSS), de la date et heure de la consultation, du bilan prescrit, du résumé et du médecin consultant. Elle associe également la consultation au patient et au médecin.

Paramètres:

request (Request): La requête HTTP contenant les informations nécessaires à la création de la consultation.

Retour:

Response: La réponse HTTP contenant un message de succès avec l'ID de la consultation créée ou un message d'erreur.

`backendapp.views.creer_ordonnance` (request, *args, **kwargs)

Crée une ordonnance pour un patient donné à partir de la consultation spécifiée.

Cette fonction crée une ordonnance pour un patient, associée à une consultation spécifique et à une liste de médicaments. Elle attend un NSS pour identifier le patient, une date de consultation pour associer la consultation, et une liste de médicaments à prescrire. Si un champ requis est manquant ou incorrect, un message d'erreur est renvoyé.

Paramètres:

request (Request): La requête HTTP contenant les données nécessaires pour créer l'ordonnance.

Retour:

Response: La réponse HTTP avec un message de succès ou d'erreur.

`backendapp.views.creer_patient` (request, *args, **kwargs)

Crée un nouveau patient à partir des données fournies.

Cette fonction permet de créer un nouveau patient en extrayant les informations envoyées dans la requête. Elle vérifie si le médecin (medecin_traitant) existe, crée un dossier patient, et associe les informations du patient. Elle s'assure également que les données du patient sont valides avant de les enregistrer.

Paramètres:

request (Request): La requête HTTP contenant les données du patient à créer.

Retour:

Response: La réponse HTTP avec les données du patient créé ou un message d'erreur en cas de problème.

`backendapp.views.get_consultations_by_nss` (request, *args, **kwargs)

Récupère les consultations d'un patient identifié par son NSS.

Cette fonction prend le NSS d'un patient, vérifie si le patient existe dans la base de données, puis récupère toutes les consultations associées à ce patient et les renvoie sous forme de réponse JSON.

Paramètres:

request (Request): La requête HTTP contenant le NSS du patient dans les données de la requête.

Retour:

Response: La réponse HTTP contenant les consultations du patient si elles sont trouvées,
ou un message d'erreur si le patient ou ses consultations ne sont pas trouvés.

`backendapp.views.get_ordonnance_by_nss_and_consultation` (request, *args, **kwargs)

Récupère l'ordonnance d'un patient identifiée par son NSS et le numéro de consultation, si le "bilan_prescrit" contient "ordonnance".

Cette fonction cherche la consultation associée au patient, vérifie si l'ordonnance est incluse dans le bilan prescrit, et retourne les informations de l'ordonnance et des médicaments associés.

Paramètres:

request (Request): La requête HTTP contenant les informations du NSS et du numéro de consultation.

Retour:

Response: La réponse HTTP avec les détails de l'ordonnance et des médicaments ou un message d'erreur si une étape échoue.

`backendapp.views.get_soins_by_nss` (request, *args, **kwargs)

Récupère les soins (dossiers de soins) pour un patient identifié par son NSS.

Cette fonction prend le NSS d'un patient, vérifie si le patient existe dans la base de données, puis récupère tous les soins associés à ce patient et les renvoie sous forme de réponse JSON.

Paramètres:

request (Request): La requête HTTP contenant le NSS du patient dans les données de la requête.

Retour:

Response: La réponse HTTP contenant les soins du patient si ils sont trouvés,
ou un message d'erreur si le patient ou ses soins ne sont pas trouvés.

`backendapp.views.login_view` (request, *args, **kwargs)

Point d'entrée pour l'authentification de l'utilisateur et la création d'un token d'authentification.

Cette fonction gère la connexion d'un utilisateur en vérifiant son nom d'utilisateur et son mot de passe. Si l'utilisateur est authentifié avec succès, elle crée un token d'authentification, et renvoie les données de l'utilisateur, y compris son rôle spécifique et les données associées (par exemple, les informations sur le patient ou le médecin).

Paramètres:

request (Request): Requête HTTP contenant les données d'identification (nom d'utilisateur et mot de passe).

Retour:

Response: Réponse HTTP contenant le token d'authentification et les informations de l'utilisateur si la connexion est réussie.

Sinon, un message d'erreur indiquant un échec de l'authentification.

`backendapp.views.rechercher_dpi_par_nss` (request, *args, **kwargs)

Recherche le dossier patient à partir du numéro de sécurité sociale (NSS).

Cette fonction permet de récupérer les informations du patient en fonction de son NSS (Numéro de Sécurité Sociale), et retourne les données du patient ainsi que son dossier médical.

Paramètres:

request (Request): La requête HTTP contenant le numéro de sécurité sociale (NSS) du patient.

Retour:

Response: La réponse HTTP contenant les données du patient et de son dossier si le NSS est valide,
ou un message d'erreur si le NSS est manquant ou que le patient n'est pas trouvé.

`backendapp.views.recuperer_bilan_biologique` (request, *args, **kwargs)

Récupère le bilan biologique pour une consultation spécifique d'un patient.

Cette fonction prend le NSS et le numéro de consultation en entrée et vérifie les conditions suivantes : - Si le patient existe. - Si le dossier patient existe. - Si la consultation existe. - Si le "bilan biologique" est prescrit. - Si un bilan biologique est disponible pour la consultation demandée.

Paramètres:

request (Request): La requête HTTP contenant les informations du NSS et du numéro de consultation.

Retour:

Response: La réponse HTTP avec les détails du bilan biologique ou un message d'erreur si une étape échoue.

`backendapp.views.recuperer_bilan_radiologique(request, *args, **kwargs)`

Récupère le bilan radiologique pour une consultation spécifique d'un patient.

Cette fonction prend le NSS et le numéro de consultation en entrée et vérifie les conditions suivantes : - Si le patient existe. - Si le dossier patient existe. - Si la consultation existe. - Si le "bilan radiologique" est prescrit. - Si un bilan radiologique est disponible pour la consultation demandée.

Paramètres:

request (Request): La requête HTTP contenant les informations du NSS et du numéro de consultation.

Retour:

Response: La réponse HTTP avec les détails du bilan radiologique ou un message d'erreur si une étape échoue.

`backendapp.views.recuperer_ordonnance(request, *args, **kwargs)`

Récupère l'ordonnance pour une consultation spécifique d'un patient.

Cette fonction prend le NSS et le numéro de consultation en entrée et vérifie les conditions suivantes : - Si le patient existe. - Si le dossier patient existe. - Si la consultation existe. - Si une ordonnance existe pour cette consultation. - Si des médicaments sont associés à l'ordonnance.

Paramètres:

request (Request): La requête HTTP contenant les informations du NSS et du numéro de consultation.

Retour:

Response: La réponse HTTP avec les détails de l'ordonnance et des médicaments ou un message d'erreur si une étape échoue.

`backendapp.views.verifier_patient_par_nss(request, *args, **kwargs)`

Vérifie si un patient existe dans la base de données en fonction de son NSS.

Cette fonction reçoit un numéro de sécurité sociale (NSS) en paramètre de la requête et vérifie si un patient avec ce NSS existe dans la base de données. Si le patient existe, elle retourne ses informations, sinon elle retourne une réponse indiquant que le patient n'a pas été trouvé.

Paramètres:

request (Request): La requête HTTP contenant le NSS en paramètre de la query.

Retour:

Response: La réponse HTTP contenant les données du patient si trouvé, ou un message d'erreur si non trouvé.

URL configuration for backendproject project.

The `urlpatterns` list routes URLs to views. For more information please see:

<https://docs.djangoproject.com/en/5.1/topics/http/urls/>

Examples: Function views

1. Add an import: `from my_app import views`
2. Add a URL to `urlpatterns`: `path("", views.home, name="home")`

Class-based views

1. Add an import: `from other_app.views import Home`
2. Add a URL to `urlpatterns`: `path("", Home.as_view(), name="home")`

Including another URLconf

1. Import the `include()` function: `from django.urls import include, path`

2. Add a URL to `urlpatterns`: `path("blog/", include("blog.urls"))`

WSGI config for backendproject project.

It exposes the WSGI callable as a module-level variable named `application`.

For more information on this file, see <https://docs.djangoproject.com/en/5.1/howto/deployment/wsgi/>

ASGI config for backendproject project.

It exposes the ASGI callable as a module-level variable named `application`.

For more information on this file, see <https://docs.djangoproject.com/en/5.1/howto/deployment/asgi/>

Index

A

`add_fieldsets` (attribut `backendapp.admin.AdministrateurAdmin`)
(attribut `backendapp.admin.InfirmierAdmin`)
(attribut `backendapp.admin.LaborantinAdmin`)
(attribut `backendapp.admin.MedecinAdmin`)
(attribut `backendapp.admin.PatientAdmin`)
(attribut `backendapp.admin.RadiologueAdmin`)
`administrateur` (attribut `backendapp.models.User`)
`Administrateur` (classe dans `backendapp.models`)
`Administrateur.DoesNotExist`
`Administrateur.MultipleObjectsReturned`
`AdministrateurAdmin` (classe dans `backendapp.admin`)
`AdministrateurSerializer` (classe dans `backendapp.serializers`)
`AdministrateurSerializer.Meta` (classe dans `backendapp.serializers`)
`adresse` (attribut `backendapp.models.User`) [1]
`antécédents` (attribut `backendapp.models.DossierPatient`) [1]
`auth_token` (attribut `backendapp.models.User`)

B

`backendapp.admin`
module
`backendapp.apps`
module
`backendapp.models`
module
`backendapp.serializers`
module
`backendapp.tests`
module
`backendapp.views`
module
`BackendappConfig` (classe dans `backendapp.apps`)
`backendproject.asgi`
module
`backendproject.urls`
module
`backendproject.wsgi`
module
`base_fields` (attribut `backendapp.admin.PatientAdminForm`)

`BILAN_CHOICES` (attribut `backendapp.models.Consultation`)
`bilan_prescrit` (attribut `backendapp.models.Consultation`) [1]
`BilanBiologique` (classe dans `backendapp.models`)
`BilanBiologique.DoesNotExist`
`BilanBiologique.MultipleObjectsReturned`
`bilanbiologique_set` (attribut `backendapp.models.DossierPatient`)
`BilanBiologiqueSerializer` (classe dans `backendapp.serializers`)
`BilanBiologiqueSerializer.Meta` (classe dans `backendapp.serializers`)
`BilanRadiologique` (classe dans `backendapp.models`)
`BilanRadiologique.DoesNotExist`
`BilanRadiologique.MultipleObjectsReturned`
`bilanradiologique_set` (attribut `backendapp.models.DossierPatient`)
`BilanRadiologiqueSerializer` (classe dans `backendapp.serializers`)
`BilanRadiologiqueSerializer.Meta` (classe dans `backendapp.serializers`)

C

`check_consultation_existence()` (dans le module `backendapp.views`)
`check_consultation_existence_bilan_biologique()` (dans le module `backendapp.views`)
`cholesterol` (attribut `backendapp.models.BilanBiologique`) [1]
`compte_rendu` (attribut `backendapp.models.BilanRadiologique`) [1]
`consultation` (attribut `backendapp.models.Ordonnance`) [1]
`Consultation` (classe dans `backendapp.models`)
`Consultation.DoesNotExist`
`Consultation.MultipleObjectsReturned`
`consultation_id` (attribut `backendapp.models.Ordonnance`)
`consultation_set` (attribut `backendapp.models.DossierPatient`)
(attribut `backendapp.models.Medecin`)
`ConsultationSerializer` (classe dans `backendapp.serializers`)
`ConsultationSerializer.Meta` (classe dans `backendapp.serializers`)
`creer_bilan_biologique()` (dans le module `backendapp.views`)

creer_bilan_radiologique() (dans le module backendapp.views)
creer_consultation() (dans le module backendapp.views)
creer_ordonnance() (dans le module backendapp.views)
creer_patient() (dans le module backendapp.views)

D

date_consultation (attribut backendapp.models.Consultation) [1]
date_examen (attribut backendapp.models.BilanBiologique) [1]
(attribut backendapp.models.BilanRadiologique) [1]
date_joined (attribut backendapp.models.User)
date_naissance (attribut backendapp.models.User) [1]
date_soin (attribut backendapp.models.Soins) [1]
declared_fields (attribut backendapp.admin.PatientAdminForm)
default_auto_field (attribut backendapp.apps.BackendappConfig)
description_soins (attribut backendapp.models.Soins) [1]
dose (attribut backendapp.models.Medicament) [1]
dossier_patient (attribut backendapp.models.BilanBiologique)
(attribut backendapp.models.BilanRadiologique)
(attribut backendapp.models.Consultation)
(attribut backendapp.models.Ordonnance)
(attribut backendapp.models.Patient) [1]
(attribut backendapp.models.Soins)
dossier_patient_id (attribut backendapp.models.BilanBiologique)
(attribut backendapp.models.BilanRadiologique)
(attribut backendapp.models.Consultation)
(attribut backendapp.models.Ordonnance)
(attribut backendapp.models.Patient)
(attribut backendapp.models.Soins)
DossierPatient (classe dans backendapp.models)
DossierPatient.DoesNotExist
DossierPatient.MultipleObjectsReturned
DossierPatientSerializer (classe dans backendapp.serializers)
DossierPatientSerializer.Meta (classe dans backendapp.serializers)

duree (attribut backendapp.models.Medicament) [1]

E

email (attribut backendapp.models.User)
etat (attribut backendapp.models.DossierPatient)

F

Faire_soin() (dans le module backendapp.views)
fields (attribut backendapp.admin.PatientAdminForm.Meta)
(attribut backendapp.serializers.AdministrateurSerializer.Meta)
(attribut backendapp.serializers.BilanBiologiqueSerializer.Meta)
(attribut backendapp.serializers.BilanRadiologiqueSerializer.Meta)
(attribut backendapp.serializers.ConsultationSerializer.Meta)
(attribut backendapp.serializers.DossierPatientSerializer.Meta)
(attribut backendapp.serializers.InfirmierSerializer.Meta)
(attribut backendapp.serializers.LaborantinSerializer.Meta)
(attribut backendapp.serializers.MedecinSerializer.Meta)
(attribut backendapp.serializers.MedicamentsSerializer.Meta)
(attribut backendapp.serializers.OrdonnanceSerializer.Meta)
(attribut backendapp.serializers.PatientSerializer.Meta)
(attribut backendapp.serializers.RadiologueSerializer.Meta)
(attribut backendapp.serializers.SoinsSerializer.Meta)
(attribut backendapp.serializers.UserSerializer.Meta)
fieldsets (attribut backendapp.admin.AdministrateurAdmin)
(attribut backendapp.admin.InfirmierAdmin)
(attribut backendapp.admin.LaborantinAdmin)
(attribut backendapp.admin.MedecinAdmin)
(attribut backendapp.admin.PatientAdmin)
(attribut backendapp.admin.RadiologueAdmin)
first_name (attribut backendapp.models.User)
form (attribut backendapp.admin.PatientAdmin)

G

`get_bilan_prescrit_display()` (méthode `backendapp.models.Consultation`)

`get_bilan_prescrit_list()` (méthode `backendapp.models.Consultation`)

`get_consultations_by_nss()` (dans le module `backendapp.views`)

`get_next_by_date_consultation()` (méthode `backendapp.models.Consultation`)

`get_next_by_date_examen()` (méthode `backendapp.models.BilanBiologique`)

(méthode `backendapp.models.BilanRadiologique`)

`get_next_by_date_joined()` (méthode `backendapp.models.User`)

`get_next_by_date_soin()` (méthode `backendapp.models.Soins`)

`get_ordonnance_by_nss_and_consultation()` (dans le module `backendapp.views`)

`get_previous_by_date_consultation()` (méthode `backendapp.models.Consultation`)

`get_previous_by_date_examen()` (méthode `backendapp.models.BilanBiologique`)

(méthode `backendapp.models.BilanRadiologique`)

`get_previous_by_date_joined()` (méthode `backendapp.models.User`)

`get_previous_by_date_soin()` (méthode `backendapp.models.Soins`)

`get_role_display()` (méthode `backendapp.models.User`)

`get_soins_by_nss()` (dans le module `backendapp.views`)

`glycemie` (attribut `backendapp.models.BilanBiologique`) [1]

`graphe` (attribut `backendapp.models.BilanBiologique`) [1]

`groups` (attribut `backendapp.models.User`)

I

`id` (attribut `backendapp.models.BilanBiologique`)

(attribut `backendapp.models.BilanRadiologique`)

(attribut `backendapp.models.Consultation`)

(attribut `backendapp.models.DossierPatient`)

(attribut `backendapp.models.Medicament`)

(attribut `backendapp.models.Ordonnance`)

(attribut `backendapp.models.Soins`)

(attribut `backendapp.models.User`)

`images` (attribut `backendapp.models.BilanRadiologique`) [1]

`infirmier` (attribut `backendapp.models.Soins`) [1]

(attribut `backendapp.models.User`)

`Infirmier` (classe dans `backendapp.models`)

`Infirmier.DoesNotExist`

`Infirmier.MultipleObjectsReturned`

`infirmier_id` (attribut `backendapp.models.Soins`)

`InfirmierAdmin` (classe dans `backendapp.admin`)

`InfirmierSerializer` (classe dans `backendapp.serializers`)

`InfirmierSerializer.Meta` (classe dans `backendapp.serializers`)

`is_active` (attribut `backendapp.models.User`)

`is_staff` (attribut `backendapp.models.User`)

`is_superuser` (attribut `backendapp.models.User`)

L

`laborantin` (attribut `backendapp.models.BilanBiologique`) [1]

(attribut `backendapp.models.User`)

`Laborantin` (classe dans `backendapp.models`)

`Laborantin.DoesNotExist`

`Laborantin.MultipleObjectsReturned`

`laborantin_id` (attribut `backendapp.models.BilanBiologique`)

`LaborantinAdmin` (classe dans `backendapp.admin`)

`LaborantinSerializer` (classe dans `backendapp.serializers`)

`LaborantinSerializer.Meta` (classe dans `backendapp.serializers`)

`last_login` (attribut `backendapp.models.User`)

`last_name` (attribut `backendapp.models.User`)

`list_display` (attribut `backendapp.admin.AdministrateurAdmin`)

(attribut `backendapp.admin.InfirmierAdmin`)

(attribut `backendapp.admin.LaborantinAdmin`)

(attribut `backendapp.admin.MedecinAdmin`)

(attribut `backendapp.admin.PatientAdmin`)

(attribut `backendapp.admin.RadiologueAdmin`)

`logentry_set` (attribut `backendapp.models.User`)

`login_view()` (dans le module `backendapp.views`)

M

`medecin` (attribut `backendapp.models.User`)

`Medecin` (classe dans `backendapp.models`)

`Medecin.DoesNotExist`

Medecin.MultipleObjectsReturned
 medecin_traitant (attribut backendapp.models.Patient) [1]
 medecin_traitant_id (attribut backendapp.models.Patient)
 MedecinAdmin (classe dans backendapp.admin)
 medecinConsultant (attribut backendapp.models.Consultation) [1]
 medecinConsultant_id (attribut backendapp.models.Consultation)
 MedecinSerializer (classe dans backendapp.serializers)
 MedecinSerializer.Meta (classe dans backendapp.serializers)
 media (propriété backendapp.admin.AdministrateurAdmin)
 (propriété backendapp.admin.InfirmierAdmin)
 (propriété backendapp.admin.LaborantinAdmin)
 (propriété backendapp.admin.MedecinAdmin)
 (propriété backendapp.admin.PatientAdmin)
 (propriété backendapp.admin.PatientAdminForm)
 (propriété backendapp.admin.RadiologueAdmin)
 Medicament (classe dans backendapp.models)
 Medicament.DoesNotExist
 Medicament.MultipleObjectsReturned
 medicament_pris (attribut backendapp.models.Soins) [1]
 medicaments (attribut backendapp.models.Ordonnance)
 MedicamentsSerializer (classe dans backendapp.serializers)
 MedicamentsSerializer.Meta (classe dans backendapp.serializers)
 model (attribut backendapp.admin.AdministrateurAdmin)
 (attribut backendapp.admin.InfirmierAdmin)
 (attribut backendapp.admin.LaborantinAdmin)
 (attribut backendapp.admin.MedecinAdmin)
 (attribut backendapp.admin.PatientAdmin)
 (attribut backendapp.admin.PatientAdminForm.Meta)
 (attribut backendapp.admin.RadiologueAdmin)
 (attribut backendapp.serializers.AdministrateurSerializer.Meta)
 (attribut backendapp.serializers.BilanBiologiqueSerializer.Meta)
 (attribut backendapp.serializers.BilanRadiologiqueSerializer.Meta)

(attribut backendapp.serializers.ConsultationSerializer.Meta)
 (attribut backendapp.serializers.DossierPatientSerializer.Meta)
 (attribut backendapp.serializers.InfirmierSerializer.Meta)
 (attribut backendapp.serializers.LaborantinSerializer.Meta)
 (attribut backendapp.serializers.MedecinSerializer.Meta)
 (attribut backendapp.serializers.MedicamentsSerializer.Meta)
 (attribut backendapp.serializers.OrdonnanceSerializer.Meta)
 (attribut backendapp.serializers.PatientSerializer.Meta)
 (attribut backendapp.serializers.RadiologueSerializer.Meta)
 (attribut backendapp.serializers.SoinsSerializer.Meta)
 (attribut backendapp.serializers.UserSerializer.Meta)

module

backendapp.admin
 backendapp.apps
 backendapp.models
 backendapp.serializers
 backendapp.tests
 backendapp.views
 backendproject.asgi
 backendproject.urls
 backendproject.wsgi

mutuelle (attribut backendapp.models.Patient) [1]

N

name (attribut backendapp.apps.BackendappConfig)
 nom (attribut backendapp.models.Medicament) [1]
 nss (attribut backendapp.models.Patient)
 numero_consultation (attribut backendapp.models.BilanBiologique) [1]
 (attribut backendapp.models.BilanRadiologique) [1]
 (attribut backendapp.models.Consultation) [1]
 numero_telephone (attribut backendapp.models.User) [1]

O

objects (attribut backendapp.models.BilanBiologique)
 (attribut backendapp.models.BilanRadiologique)
 (attribut backendapp.models.Consultation)
 (attribut backendapp.models.DossierPatient)
 (attribut backendapp.models.Medicament)
 (attribut backendapp.models.Ordonnance)
 (attribut backendapp.models.Soins)
 observation_etat_patient (attribut backendapp.models.Soins) [1]
 ordering (attribut backendapp.admin.AdministrateurAdmin)
 (attribut backendapp.admin.InfirmierAdmin)
 (attribut backendapp.admin.LaborantinAdmin)
 (attribut backendapp.admin.MedecinAdmin)
 (attribut backendapp.admin.PatientAdmin)
 (attribut backendapp.admin.RadiologueAdmin)
 OrdonnaceSerializer (classe dans backendapp.serializers)
 OrdonnaceSerializer.Meta (classe dans backendapp.serializers)
 ordonnance (attribut backendapp.models.Medicament)
 Ordonnance (classe dans backendapp.models)
 Ordonnance.DoesNotExist
 Ordonnance.MultipleObjectsReturned
 ordonnance_id (attribut backendapp.models.Medicament)
 ordonnance_set (attribut backendapp.models.Consultation)
 (attribut backendapp.models.DossierPatient)
 OrdonnanceModelTestCase (classe dans backendapp.tests)

P

password (attribut backendapp.models.User)
 patient (attribut backendapp.models.DossierPatient)
 (attribut backendapp.models.User)
 Patient (classe dans backendapp.models)
 Patient.DoesNotExist
 Patient.MultipleObjectsReturned
 PatientAdmin (classe dans backendapp.admin)
 PatientAdminForm (classe dans backendapp.admin)
 PatientAdminForm.Meta (classe dans backendapp.admin)
 patients (attribut backendapp.models.Infirmier)

(attribut backendapp.models.Laborantin)
 (attribut backendapp.models.Medecin)
 (attribut backendapp.models.Radiologue)
 PatientSerializer (classe dans backendapp.serializers)
 PatientSerializer.Meta (classe dans backendapp.serializers)
 pression_arterielle (attribut backendapp.models.BilanBiologique) [1]

R

radiologue (attribut backendapp.models.BilanRadiologique) [1]
 (attribut backendapp.models.User)
 Radiologue (classe dans backendapp.models)
 Radiologue.DoesNotExist
 Radiologue.MultipleObjectsReturned
 radiologue_id (attribut backendapp.models.BilanRadiologique)
 RadiologueAdmin (classe dans backendapp.admin)
 RadiologueSerializer (classe dans backendapp.serializers)
 RadiologueSerializer.Meta (classe dans backendapp.serializers)
 rechercher_dpi_par_nss() (dans le module backendapp.views)
 recuperer_bilan_biologique() (dans le module backendapp.views)
 recuperer_bilan_radiologique() (dans le module backendapp.views)
 recuperer_ordonnance() (dans le module backendapp.views)
 resume (attribut backendapp.models.Consultation) [1]
 role (attribut backendapp.models.User)
 ROLE_CHOICES (attribut backendapp.models.User)

S

save_model() (méthode backendapp.admin.AdministrateurAdmin)
 (méthode backendapp.admin.InfirmierAdmin)
 (méthode backendapp.admin.LaborantinAdmin)
 (méthode backendapp.admin.MedecinAdmin)
 (méthode backendapp.admin.PatientAdmin)
 (méthode backendapp.admin.RadiologueAdmin)
 search_fields (attribut backendapp.admin.AdministrateurAdmin)
 (attribut backendapp.admin.InfirmierAdmin)

(attribut backendapp.admin.LaborantinAdmin)
(attribut backendapp.admin.MedecinAdmin)
(attribut backendapp.admin.PatientAdmin)
(attribut backendapp.admin.RadiologueAdmin)
setUp() (méthode backendapp.tests.OrdonnanceModelTestCase)
Soins (classe dans backendapp.models)
Soins.DoesNotExist
Soins.MultipleObjectsReturned
soins_set (attribut backendapp.models.DossierPatient)
SoinsSerializer (classe dans backendapp.serializers)
SoinsSerializer.Meta (classe dans backendapp.serializers)

T

telephone_urgence (attribut backendapp.models.Patient) [1]
test_dossier_patient_association() (méthode backendapp.tests.OrdonnanceModelTestCase)
test_ordonnance_creation() (méthode backendapp.tests.OrdonnanceModelTestCase)

U

User (classe dans backendapp.models)
User.DoesNotExist
User.MultipleObjectsReturned
user_permissions (attribut backendapp.models.User)
user_ptr (attribut backendapp.models.Administrateur)
(attribut backendapp.models.Infirmier)
(attribut backendapp.models.Laborantin)
(attribut backendapp.models.Medecin)
(attribut backendapp.models.Patient)
(attribut backendapp.models.Radiologue)
user_ptr_id (attribut backendapp.models.Administrateur)
(attribut backendapp.models.Infirmier)
(attribut backendapp.models.Laborantin)
(attribut backendapp.models.Medecin)
(attribut backendapp.models.Patient)
(attribut backendapp.models.Radiologue)
username (attribut backendapp.models.User)
UserSerializer (classe dans backendapp.serializers)
UserSerializer.Meta (classe dans backendapp.serializers)

V

verifier_patient_par_nss() (dans le module backendapp.views)

Index des modules Python

b

backendapp

[backendapp.admin](#)

[backendapp.apps](#)

[backendapp.models](#)

[backendapp.serializers](#)

[backendapp.tests](#)

[backendapp.views](#)

backendproject

[backendproject.asgi](#)

[backendproject.urls](#)

[backendproject.wsgi](#)